Distributed Task Scheduler

Submitted By

Student Name	Student ID
Mubas Sera Noon	0242310005101051
Md. Ibna Labid	0242310005101180
Md Rizwan Karim Lam	0242310005101608
Md Ahsanur Rahman Nakib	0242310005101611
Fahim Montasir	0242310005101748

MINI LAB PROJECT REPORT

This Report Presented in Partial Fulfillment of the course

CSE324: Operating System Lab in the

Computer Science and Engineering Department

DAFFODIL INTERNATIONAL UNIVERSITY

Dhaka, Bangladesh

December 14 , 2025

DECLARATION

We hereby declare that this lab project has been done by us under the supervision of Md Mehefujur Rahman Mubin, Lecturer, Department of Computer Science and Engineering, Daffodil International University. We also declare that neither this project nor any part of this project has been submitted elsewhere as lab projects.

Submitted To:

Md Mehefujur Rahman Mubin,
Lecturer
Department of Computer Science and Engineering
Daffodil International University.

Submitted by

Rizwan Karim Lam
ID:0242310005101608
Dept. of CSE, DIU

Fahim Montasir
ID:0242310005101748
Dept. of CSE, DIU

Ahsanur Rahman Nakib
ID:0242310005101611
Dept. of CSE, DIU

Mubas Sera Noon
ID:0242310005101051
Dept. of CSE, DIU

Md. Ibna Labid
ID:0242310005101180
Dept. of CSE, DIU

COURSE & PROGRAM OUTCOME

The following course have course outcomes as following:

Table 1: Course Outcome Statements

CO's	Statements
CO1	Define and analyze task scheduling concepts and system resources
CO2	Apply operating system scheduling algorithms in problem solving
CO3	Analyze system behavior using priority-based scheduling
CO4	Design and implement a real-world scheduling solution

Table 2: Mapping of CO, PO, Blooms, KP and CEP

CO	PO	Blooms	KP	CEP
CO1	PO1	C1, C2	KP3	EP1
CO2	PO2	C3	KP3	EP1, EP3
CO3	PO3	C4	KP4	EP2
CO4	PO3	C6	KP4	EP3

The mapping justification of this table is provided in section **4.3.1**, **4.3.2** and **4.3.3**.

Table of Contents

Chapter 1

Introduction

This chapter presents an overview of the project, including background, motivation, objectives, feasibility, gap analysis, and expected outcomes.

1.1 Introduction

In modern computing systems, executing multiple tasks concurrently is a fundamental requirement. Operating systems manage this process through task scheduling mechanisms. This project implements a Distributed Task Scheduler that simulates how tasks are scheduled based on priority, CPU usage, memory requirements, storage availability, and execution time.

1.2 Motivation

Task scheduling plays a vital role in operating systems such as Linux and Windows. The main motivations behind this project are:

- To understand operating system scheduling concepts practically
- To learn how preemptive priority scheduling works
- To gain hands-on experience with resource management and concurrency

1.3 Objectives

Task scheduling plays a vital role in operating systems such as Linux and Windows. The main motivations behind this project are:

- To understand operating system scheduling concepts practically
- To learn how preemptive priority scheduling works
- To gain hands-on experience with resource management and concurrency

1.4 Feasibility Study

Modern operating systems use advanced scheduling algorithms to manage processes efficiently.

This project is feasible as it is implemented using Python, threading, and priority queues, which are well-suited for simulating concurrent task scheduling behavior.

1.5 Gap Analysis

Most academic scheduling examples are static and theoretical. This project bridges the gap by introducing:

- Dynamic task arrival
- Real-time task execution and preemption
- Practical resource constraint handling

1.6 Project Outcome

The outcome of this project is a fully functional Distributed Task Scheduler Simulation that demonstrates real-world operating system scheduling behavior in an educational environment.

Chapter 2

Proposed Methodology/Architecture

This chapter describes the system requirements, architecture, and design methodology.

2.1 Requirement Analysis & Design Specification

The following requirements are necessary to develop the system:

- Python Programming Language
- Threading and Synchronization Mechanisms
- Priority Queue (Heap)
- Command-Line Interface (CLI)

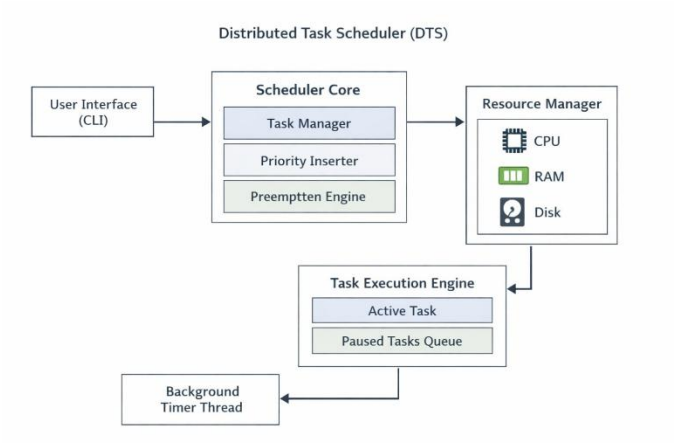
2.1.1 Overview

The system architecture consists of three major components:

1. Running Tasks
2. Ready Queue
3. Resource Manager (CPU, RAM, Storage)

2.1.2 Proposed Methodology/ System Design

Figure 2.1: This is a sample diagram



2.1.3 UI Design

The user interface is terminal-based and displays:

- Current running tasks
- Ready queue tasks
- Available system resources

2.2 Overall Project Plan

The project was completed in the following phases:

1. Problem Analysis
2. System Design
3. Implementation
4. Testing
5. Documentation

Chapter 3

Implementation and Results

The system is implemented using Python threading, priority queues, and synchronization mechanisms to manage concurrent task execution efficiently. Performance analysis shows that high-priority tasks are executed promptly while minimizing resource starvation and maintaining system stability.

3.1 Implementation

The system is implemented using Python threading to handle concurrent task execution.

A priority queue is used to manage the ready queue, ensuring that higher-priority tasks are scheduled first.

3.2 Performance Analysis

- Higher-priority tasks are executed with minimal delay
- System resources are utilized efficiently
- The scheduler maintains system stability under multiple tasks

3.3 Results and Discussion

The results demonstrate that the scheduler effectively mimics real operating system behavior, including preemption, priority handling, and resource management.

Chapter 4

Engineering Standards and Mapping

This chapter evaluates project's engineering standards, emphasizing sustainability, social impact, and ethical considerations.It also maps the project outcomes to DIU program outcomes, demonstrating structured problem-solving and engineering practices.

4.1 Impact on Society, Environment and Sustainability

Efficient task scheduling improves system performance and reduces energy consumption, contributing to sustainable computing practices.

4.2 Project Management and Team Work

The project was completed individually using proper planning, time management, and systematic development practices.

4.3 Complex Engineering Problem

This project addresses complex engineering challenges such as:

- Concurrent task execution
- Resource conflicts
- Priority-based decision making.These characteristics qualify it as a complex engineering problem.

4.3.1 Mapping of Program Outcome

In this section, provide a mapping of the problem and provided solution with targeted Program Outcomes (PO's).

Table 4.1: Justification of Program Outcomes

PO's	**Justification**
PO1	Applied OS concepts and scheduling theory
PO2	Analyzed task conflicts and resource limits
PO3	Designed and implemented scheduler
P04	Investigated performance behavior

4.3.2 Complex Problem Solving

In this section, provide a mapping with problem solving categories. For each mapping add subsections to put rationale (Use Table 4.2). For P1, you need to put another mapping with knowledge profile and rational thereof.

Table 4.2: Mapping with complex problem solving.

EP1 Dept of Knowledge	**EP2** Range of Conflicting Requirements	**EP3** Depth of Analysis	**EP4** Familiarity of Issues	**EP5** Extent of Applicable Codes	**EP6** Extent Of Stakeholder Involvement	**EP7** Inter-dependence
✔	✔	✔				

4.3.3 Engineering Activities

In this section, provide a mapping with engineering activities. For each mapping add subsections to put rationale (Use Table 4.3).

Table 4.3: Mapping with complex engineering activities.

EA1 Range of resources	**EA2** Level of Interaction	**EA3** Innovation	**EA4** Consequences for society and environment	**EA5** Familiarity
✔	✔	✔		

Chapter 5

Conclusion

The project successfully develops a Distributed Task Scheduler that simulates priority-based and preemptive scheduling techniques.Although effective, the system is limited to simulation and operates through a command-line interface without real hardware control.Future improvements may include a graphical interface, real-time performance visualization, and network-based distributed scheduling.

5.1 Summary

This project successfully implements a Distributed Task Scheduler that simulates priority-based and preemptive scheduling techniques.

5.2 Limitation

- The system is a simulation and does not control real hardware
- The scheduler is limited to a command-line interface

5.3 Future Work

Future enhancements may include:

- Graphical User Interface (GUI)
- Real-time performance visualization
- Network-based distributed task scheduling

Reference

[1] Jon Kleinberg and Eva Tardos, Algorithm Design, Pearson Education